

29.9% of returns aren't genuine. They carry **\$4.64M** of refund dollars — and now we know which ones.

A retailer's return-audit team reviews every disputed return the same way — spreadsheet speed, inconsistent criteria. We rebuilt **60,000 orders** into a PySpark/MLlib pipeline that triages each one into legitimate, policy abuse, fraud, or wardrobing, priced what's actually at stake, and separately segmented customers to surface the product-quality problem no fraud label can see.

NON-LEGITIMATE REFUND EXPOSURE

\$4.64M

43.8% of all refund dollars requested, from just **29.9%** of orders — bad actors skew toward higher-value claims

CLASSIFIER PERFORMANCE (TEST SET)

0.999

weighted F1, Random Forest — synthetic-dataset caveat on slide 6, not a production guarantee

HIDDEN IN THE "LEGITIMATE" MAJORITY

38%

of returns in the largest, 96%-legitimate customer segment are still quality-issue driven

60,000 orders · 35 features, Kaggle e-commerce return-abuse dataset · Fraudulent Return claims average **\$434.73** vs. **\$141.14** for legitimate returns — a **3.1x** gap · two independent PySpark MLlib approaches: multi-class classification (Random Forest vs. Logistic Regression) and K-Means clustering · this dataset is synthetic, so treat 0.999 F1 as a pipeline proof, not a production number (more on slide 6).

One review queue, *three different problems* wearing the same form.

A return-audit team currently reviews every disputed return the same way. It has no systematic way to tell whether a spike in returns is fraud, abuse of a lenient policy, or a genuine quality defect — three problems that need three different owners, currently indistinguishable in a spreadsheet.

01 · CUSTOMERS RETURN

Most returns are genuine

Defective items, wrong fit, changed mind — the normal cost of running an e-commerce store.

70.1% legitimate

02 · SOME ABUSE IT

Policy abuse, fraud, wardrobing

Exploiting lenient terms, deceptive claims, or wearing-and-returning — three distinct behaviors.

29.9% of orders

03 · ALL LOOK THE SAME

One manual-review process

Every disputed return gets identical treatment today, regardless of which of the three it actually is.

1 review queue

04 · THE WRONG FIX

Blanket policy tightening

Punishes the 70% of legitimate customers to catch the fraction who abuse the system.

wrong lever

THE QUESTION THIS PROJECT ANSWERS

Which returns are fraud, which are policy abuse, and which are genuine quality failures — and what does each one actually cost?

Three sub-problems, one dataset: **return fraud** → classification on abuse_type; **product quality** → return-reason patterns cut by cluster and category; **return risk** → lifetime order/return history per customer.

WHY THIS IS A TRIAGE PROBLEM, NOT A POLICY PROBLEM

The highest-value lever is routing each return to the right team fast — not tightening policy across the board, which would punish the 70% of customers who did nothing wrong. **Three different interventions, three different owners**, is the recommendation this whole pipeline builds toward.

A realistic imbalance — and the dollars don't split the way the order counts do.



■ Legitimate — 70.1% (42,060 orders, \$5.94M requested) ■ Policy Abuser — 12.0% (7,192, \$0.76M) ■ Fraudulent Return — 10.2% (6,112, \$2.66M) ■ Wardrobing — 7.7% (4,636, \$1.22M)

Behavioral signals

Photo evidence, tracking validity, address changes, refund-to-different-account, multiple accounts, dispute history

Customer history

Lifetime orders/returns, return rate, account age, loyalty segment

Context

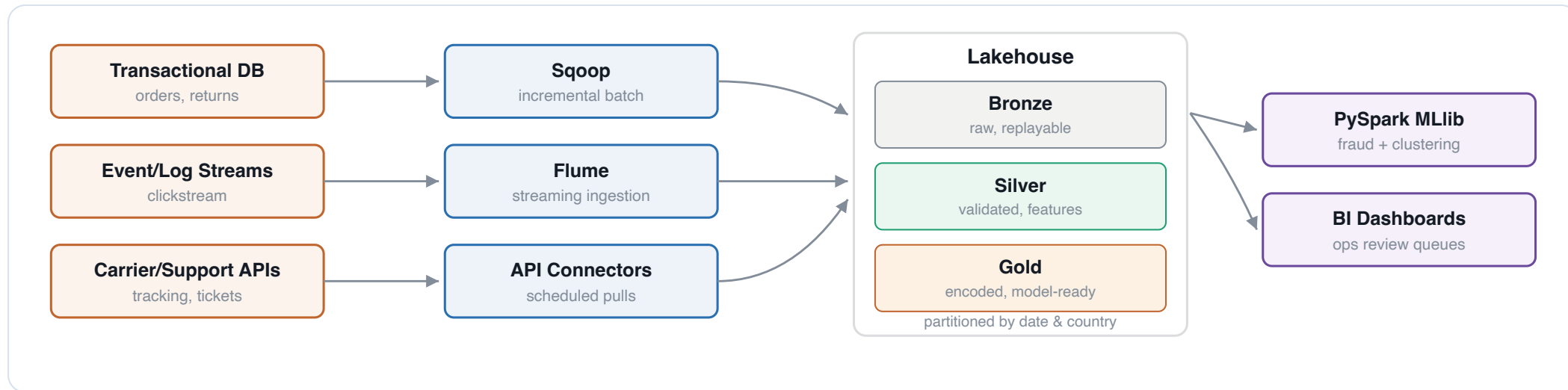
12 product categories, 12 return reasons, 8 countries, payment method, shipping carrier

VERIFIED — CHECKED PROGRAMMATICALLY, NOT ASSUMED

0 missing values across all 35 columns, **0 duplicate** order_id, **0 logical violations** (no return-before-order dates, no returns exceeding lifetime orders, no implausible ages) — confirmed in notebooks/01_data_engineering.ipynb §2 before anything downstream trusted this data.

Sqoop / Flume / API ingestion → *Bronze/Silver/Gold lakehouse* → PySpark MLlib.

This notebook runs directly on the static Kaggle CSV as a practical stand-in — the diagram below is the production-scale design the Silver→Gold transformation logic is built to generalize to.



WHY BRONZE/SILVER/GOLD

Bronze keeps a permanent, replayable raw record. Silver is cleaned and feature-engineered once, reused by both models. Gold is the only layer anything ML touches.

EXPLICIT SCHEMA, NOT INFERRED

The notebook loads the CSV with an explicit StructType — never inferSchema in production, which forces a full extra scan and can silently mis-type columns on drift.

CAP OUTLIERS, DON'T DELETE

Extreme values are often the fraud signal itself. 8 continuous fields IQR-capped at the 1st/99th percentile — preserving row count and the extremeness of the behavior.

Engineered features *validated* before they ever reach a model.

FEATURE	SERVES
fraud_signal_score	Return fraud
return_to_order_ratio	Return risk
refund_to_order_value_ratio	Return fraud
is_quality_issue_reason	Product quality
high_value_no_evidence	Return fraud
Outliers capped at 1st/99th percentile, not deleted — extreme values are often the fraud signal itself, not noise to remove.	

VERIFIED — PRINTED BY THE RUN, NOT CLAIMED		
Mean fraud_signal_score : 2.73 for Fraudulent Return vs. 0.84 for Legitimate. Mean return_to_order_ratio : 0.47 vs. 0.05. Clear separation confirmed <i>before</i> trusting these features for modeling — not assumed. notebooks/01_data_engineering.ipynb §4.		
	FRAUDULENT RETURN	LEGITIMATE
fraud_signal_score	2.73	0.84
return_to_order_ratio	0.47	0.05

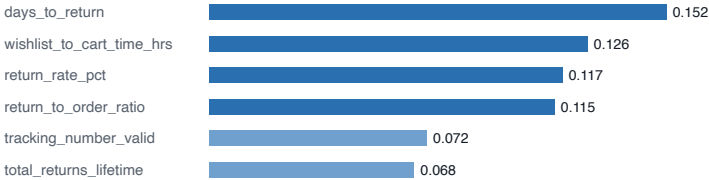
Random Forest vs. Logistic Regression on *return-fraud type*.

Spark MLlib's `GBTClassifier` only supports binary classification — not usable for this 4-class target — so Random Forest, which natively supports multi-class, is the primary model. Class-weighted training (inverse frequency) handles the 70/12/10/8 imbalance; evaluation reports per-class precision/recall/F1, not headline accuracy.

MODEL	WEIGHTED PRECISION	WEIGHTED RECALL	WEIGHTED F1
Logistic Regression (baseline)	0.998	0.998	0.998
Random Forest (final)	0.999	0.999	0.999

200 trees, maxDepth=10, class-weighted. Train 45,112 / test 14,888 rows, 75/25 split, seed=42.

TOP FEATURE IMPORTANCES



Engineered ratios rank alongside raw behavioral fields — the payoff of Phase 2.

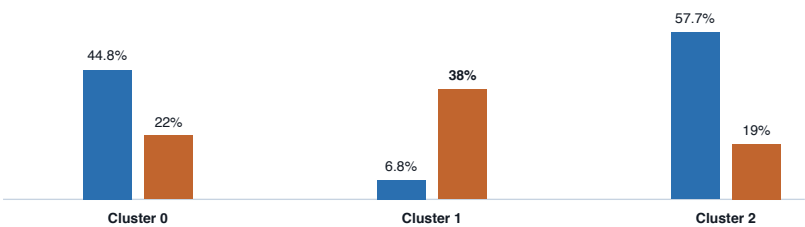
Honest note: this is a Kaggle synthetic dataset, so 0.999 F1 is higher than we'd expect in production — feature importance is spread across ~15 features with no single leak, but a realistic target on live data is **80–90% F1**. We treat this model as proof of the pipeline, not a performance guarantee. Full discussion in `docs/fraud-classification-results.md`.

APPROACH 2 — CLUSTERING

K-Means finds 3 behavioral segments *the fraud label can't see.*

An independent, unsupervised lens: features deliberately exclude the fraud-flag columns used by the classifier, so clustering surfaces behavior/quality patterns a supervised label alone can't. k=3 selected by silhouette score over k=2..6.

CLUSTER	N	AVG RETURN RATE	AVG ORDER VALUE	% QUALITY-ISSUEDOMINANT MIX
0 — High-value fraud-risk	6,646	44.8%	\$484	22% Fraudulent Return-heavy
1 — Core legitimate base	43,829	6.8%	\$157	38% 96% Legitimate
2 — Frequent policy-abuse	9,525	57.7%	\$129	19% Policy Abuser-heavy



KEY INSIGHT

Cluster 1 has by far the **lowest** return rate but the **highest** quality-issue share — the two metrics move in opposite directions, which is exactly why a single blended "return rate" KPI hides this segment's problem from whoever owns it today.

This is a **QA/vendor problem**, not a fraud problem — cross-reference dominant product_category within Cluster 1 against return_reason to target the escalation.

Three interventions, three owners — *ship the free one first.*

SHIP THIS FIRST, NO ML REQUIRED

Require photo evidence above a value threshold. The "high-value item + no photo evidence" pattern alone touches **\$1.88M** of non-legitimate refund exposure across **4,673 orders** — a policy rule, live immediately, independent of the classifier's rollout timeline.

01 Deploy the fraud classifier as a triage score, not an auto-block.

Route the highest fraud_signal_score decile + Cluster 0 to manual review; auto-approve the lowest-risk tail. **Re-threshold asymmetrically** — a false "Legitimate → Fraudulent Return" flag costs more in trust than a slow review costs in ops time.

OWNER · FRAUD & RISK

02 Give Cluster 1's 38% quality-issue rate to QA, not fraud.

Cross-reference dominant product category × return reason within the core-legitimate segment to target vendor escalations — **73% of customers**, near-zero fraud, but the highest quality complaint rate of any segment.

OWNER · QA / VENDOR MANAGEMENT

03 Tighten policy — don't investigate — for Cluster 2.

High order/return volume, low order value, Policy-Abuser-dominated. A shorter return window or restocking fee targets this pattern more precisely than a fraud investigation would.

OWNER · CUSTOMER POLICY



THANK YOU

Thank you. Questions *welcome.*

We'd rather be asked about the caveats than the number —
\$4.64M of non-legitimate refund exposure, **\$1.88M** of it
reachable today with a single no-ML policy rule.

Subham Ranjan · A017

Shriya · A023

Abhinav Kumar · A035

Dev Vagrecha · A049

Samruddhi Pradhan · A054

Yatharth Vij · A072

IN THE REPOSITORY

notebooks/ — two executable
PySpark notebooks, Bronze to
models

WRITTEN UP

docs/ — per-model results, each
with its own weaknesses section

THE DELIVERABLES

deliverables/ — this deck and the
full consulting report, HTML + PDF

REPRODUCIBILITY

seed=42 throughout · every number
here read from the notebooks' own
output

Dataset: `ecommerce_return_abuse_dataset.csv` (Kaggle) · references, AI-usage disclosure and team contributions in the consulting report's Appendix.